Final Report - Protein Interaction Prediction

CMSC 435 Group Project

Column Crushers (Connor Fair, Charles Wilson)

2025-12-15

The Prediction System

Feature Extraction

The following features were extracted from each protien sequence using biopython to be processed by our model

- **Amino acid composition (20 features)**
  - Frequency of each of the 20 standard amino acids (A, C, D, E, F, G, H, I, K, L, M, N, P, Q, R, S, T, V, W, Y)
  - Each feature represents the percentage of that amino acid in the protein sequence
  - Features sum to 100% for each protein sequence
- **Length of protien**
  - Length of protien sequences
- **Molecular weight of the protien**
  - The molecular weight of the protien
- **Gravy**
  - Describes the hydrophobicity of the protien: positive repels water and negative attracts water
- **Helix, Turn & Sheet**
  - The fraction of protiens that are structured as helix, turn & sheet
- **Positive-Negative ratio**
  - the positive to negative amino acid ratio in the protien
- **Amino acid group counts**
  - The number of protiens that are non-polar, polar(no charge), polar(positive), polar(negative)

Models

Our starategy for this assignment was to get as many models as possible, fine-tune each of them to either be really good at some metrics or decent with all metrics them combine all of them into either a voting or stacking ensemble to hopefully get the benifits of each individual model and reduce their weaknesses, although this came at the cost of time spent adjusting the features we were working with. Each model that we used was built in a scikit-learn pipeline with is own sampler and preprocesser so that the best ones for each model could be used when all the models are combined into a voter or stacker classifier.

The training process for each model is listed below:

1. A baseline model is created with default parameters and no preprocessors or samplers so that all subsequent tests had a baseline to ensure improvement
2. Using GridSearchCV, we tested all combinations of preprocessors and samplers on the model with default parameters to find the best combination

- samplers: (none, random over sampler, ADASYN, SMOTE, tomek links, random under sampler)

- preprocessors: (none, standard scaler, min-max scaler, quantile transformer)

3. Using the best found combination of samplers and preprocessors, the hyper-parameters for each model were tested with GridSearchCV

The models that were used are listed below along with their respective samplers and preprocessors.

Machine Learning Models Used

Model	Sampler	Preprocessor	Classifier	Key Hyperparameters
MLP	OverSampler	StandardScaler + dtype_fix	NeuralNetClassifier	max_epochs=40; lr=0.001; activation=ELU; units=20
KNN	TomekLinks	StandardScaler	KNN	n_neighbors=8; weights=distance
Linear SVM	OverSampler	MinMaxScaler	SGDClassifier	loss=hinge; penalty=elasticnet; n_jobs=-1
Logistic Regression	OverSampler	MinMaxScaler	SGDClassifier	loss=log_loss; penalty=L1; alpha=0.001; class_weight=balanced
GB Model 1	OverSampler	QuantileTransformer	GradientBoosting	lr=0.001; depth=1; estimators=25
GB Model 2	OverSampler	QuantileTransformer	GradientBoosting	lr=0.1; depth=3; estimators=40
SVM (RBF)	OverSampler	StandardScaler	SVC	C=1.0; gamma=scale; probability=True

The Results

For our final models, there were 3 ensemble models created: - **Hard Voting**: The model predictions are used as votes for a particular class, and the class with the most votes is chosen. - **Soft Voting**: The models probabilities for each class are used, and the class with the largest sum is the final class. This voting system requires the participating models to have class probabilities, so we had to leave out linear SVM, SVM and logistic regression from this classifier. - **Stacker**: Takes the output of all the previous models as input to a final model. In our case it was a gradient boosting model with 50 estimators.

Below is the table containing our results and the baseline given at the start of the project:

Design 1 is the hard voting model, design 2 is the soft voting model and design 3 is the stacking model

Outcome	Quality Measure	Baseline	Design 1	Design 2	Design 3	Best Design
DNA	Sensitivity	6.9	17.74	32.48	47.56	32.48
	Specificity	99.3	88.91	97.48	99.49	97.48
	Accuracy	95.2	87.24	94.30	95.51	94.30
	MCC	**0.132**	**0.2480**	**0.2618**	**0.2029**	**0.2618**
RNA	Sensitivity	39.6	43.11	71.39	72.27	71.39
	Specificity	98.9	94.96	98.73	98.86	98.73
	Accuracy	95.3	92.11	99.23	99.45	99.23
	MCC	**0.501**	**0.4738**	**0.5774**	**0.5615**	**0.5774**
DRNA	Sensitivity	4.5	1.30	4.00	0.00	4.00
	Specificity	100.0	92.24	99.45	99.70	99.45
	Accuracy	99.7	92.91	95.84	95.77	95.84

Outcome	Quality Measure	Baseline	Design 1	Design 2	Design 3	Best Design
	MCC	**0.122**	**0.0616**	**0.0568**	**-0.0027**	**0.0568**
nonDRNA	Sensitivity	98.6	96.86	93.70	92.70	93.70
	Specificity	29.8	79.06	45.73	34.94	45.73
	Accuracy	91.3	77.12	90.78	91.71	90.78
	MCC	**0.428**	**0.3800**	**0.4677**	**0.4691**	**0.4677**
averageMCC	—	**0.296**	**0.2909**	**0.3409**	**0.3077**	**0.3409**
accuracy4labels	–	90.8	74.69	90.07	91.22	90.07

The confusion matrix for the baseline

Actual Predicted	**DNA**	**RNA**	**DRNA**	**nonDRNA**
DNA	27	20	0	344
RNA	21	207	1	294
DRNA	0	2	1	19
nonDRNA	36	71	1	7,751

The confusion matrix for our best result

Actual Predicted	**DNA**	**RNA**	**DRNA**	**nonDRNA**
DNA	102	29	6	254
RNA	20	262	4	237
DRNA	3	0	2	17
nonDRNA	189	76	38	7,556

Conlcusion

When comparing our results to the baseline:

- I significantly improved sensitivity for DNA and RNA, DRNA is around the same and nonDRNA is worse
- Average MCC improved a decent amount, same for DNA, RNA and nonDRNA
- DRNA performance is generally worse

This experience taught me a lot about the importance of extracting features from data, understanding the limits of what my computer can handle and having a consistent way to pick model parameters. The 2 largest hurdles to overcome in this project were getting useful features from the data and picking the best hyperparameters, samplers and preprocessors for each model. The biggest example of the issue with hyperparameters came from the MLP which was implemented in pytorch and put into skorch for compatibility with scikit-learn. This model was very difficult to narrow down the best parameters for due to the sheer number of parameters possible, so I had to put limits on them. The advantage to our approach to this project was the ability to combine the strengths of our models together, however this came at the cost of time that could have been spent finding better features for our models to work with. We believe this was the best approach to take due to our lack of background in biology.

We think our model is slightly better than the baseline for the most part, although it is slightly worse at DRNA in general. We predict our model to be decent at DNA and RNA on the test dataset, but not great at DRNA or nonDRNA.